

Logs and Metrics

Arturo Silvelo

Try New Roads

Logs

Logs

El comando `docker logs` permite visualizar la salida generada por los procesos que se ejecutan dentro de un contenedor. Por defecto, muestra todo lo que la aplicación escribe en los flujos estándar de salida y error (STDOUT y STDERR), tal como lo veríamos si ejecutáramos el proceso en una terminal. Sin embargo, la utilidad de este comando depende de cómo la aplicación gestione sus propios logs. Si la aplicación escribe sus registros en archivos internos o si se utiliza un driver de logs externo, es posible que `docker logs` no muestre información relevante.

Logging drivers

Docker ofrece varios mecanismos para gestionar y recolectar los logs de los contenedores y servicios, conocidos como `logging drivers` o controladores de logs. Estos controladores permiten enviar los registros a distintos destinos.

- **json-file**: Es el driver por defecto. Guarda los logs en archivos JSON en el host.
- **syslog**: Envía los logs al sistema syslog del host o a un servidor remoto.
- **fluentd**: Permite enviar los logs a un agregador Fluentd.
- **awslogs**: Envía los logs a Amazon CloudWatch Logs.
- **gelf**: Envía los logs a sistemas compatibles con Graylog Extended Log Format.
- **none**: Desactiva el registro de logs para el contenedor.

Configuración

Puedes configurar el comportamiento de los logs de manera global para todos los contenedores editando el archivo `/etc/docker/daemon.json`.

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3",
    "labels": "production_status",
    "env": "os,customer"
  }
}
```

- Ejecución (Sin logs)

```
docker run -it --rm --init --log-driver none --name app ghcr.io/trynewroads/docker-course-advance
```

- Verificación

```
$docker logs app  
Error response from daemon: configured logging driver does not support reading
```

- Ejecución (Json-file)

```
docker run -it --rm --init --log-opt labels=environment --log-opt env=os --label environment=production -e os=ubuntu --name app ghcr.io/trynewroads/docker-course-advance
```

- Verificación

```
sudo cat /var/lib/docker/containers/<container_id>/<container_id>-json.log | jq
```

GELF

GELF

El driver **GELF** (Graylog Extended Log Format) permite enviar los logs de los contenedores Docker a sistemas de gestión de logs compatibles con Graylog, como Graylog, Logstash o Fluentd. GELF es útil para centralizar, analizar y visualizar logs de múltiples contenedores en una sola plataforma.

- Permite enviar logs en formato estructurado (JSON) a través de UDP o TCP.
- Facilita la integración con herramientas de monitoreo y análisis.
- Soporta la inclusión de metadatos como etiquetas y variables de entorno.

Opción	Requerido	Descripción	Ejemplo
gelf-address	Sí	Dirección del servidor GELF (UDP/TCP y puerto).	<code>--log-opt gelf-address=udp://192.168.0.42:12201</code>
env	No	Lista de variables de entorno a incluir.	<code>--log-opt env=os,customer</code>
labels	No	Lista de labels a incluir como campos extra.	<code>--log-opt labels=production_status,geo</code>

```
name: log
# For DataNode setup, graylog starts with a preflight UI, this is a change from just using OpenSearch/Elasticsearch.
# Please take a look at the README at the top of this repo or the regular docs for more info.

services:
  # MongoDB: https://hub.docker.com/_/mongo/
  mongodb:
    image: "mongo:7.0"
    restart: "on-failure"
    networks:
      - log
    volumes:
      - "mongodb_data:/data/db"
      - "mongodb_config:/data/configdb"

  # For DataNode setup, graylog starts with a preflight UI, this is a change from just using OpenSearch/Elasticsearch.
  # Please take a look at the README at the top of this repo or the regular docs for more info.
  # Graylog Data Node: https://hub.docker.com/r/graylog/graylog-datanode

  # ⚠ Make sure this is set on the host before starting:
  # echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
  # sudo sysctl -p
  datanode:
    image: "${DATANODE_IMAGE:-graylog/graylog-datanode:6.3}"
    hostname: "datanode"
    environment:
      GRAYLOG_DATANODE_NODE_ID_FILE: "/var/lib/graylog-datanode/node-id"
      # GRAYLOG_DATANODE_PASSWORD_SECRET and GRAYLOG_PASSWORD_SECRET MUST be the same value
      GRAYLOG_DATANODE_PASSWORD_SECRET: "${GRAYLOG_PASSWORD_SECRET:?Please configure GRAYLOG_PASSWORD_SECRET in the .env file}"
      GRAYLOG_DATANODE_MONGODB_URI: "mongodb://mongodb:27017/graylog"
    ulimits:
      memlock:
        hard: -1
        soft: -1
      nofile:
        soft: 65536
        hard: 65536
    ports:
      - "8999:8999/tcp" # DataNode API
      - "9200:9200/tcp"
      - "9300:9300/tcp"
    networks:
      - log
    volumes:
      - "graylog-datanode:/var/lib/graylog-datanode"
    restart: "on-failure"

  # Graylog: https://hub.docker.com/r/graylog/graylog-enterprise
  graylog:
    hostname: "server"
    image: "${GRAYLOG_IMAGE:-graylog/graylog:6.3}"
    depends_on:
      mongodb:
        condition: "service_started"
      datanode:
        condition: "service_started"
    entrypoint: "/usr/bin/tini -- /docker-entrypoint.sh"
    environment:
      GRAYLOG_NODE_ID_FILE: "/usr/share/graylog/data/data/node-id"
      # GRAYLOG_DATANODE_PASSWORD_SECRET and GRAYLOG_PASSWORD_SECRET MUST be the same value
      GRAYLOG_PASSWORD_SECRET: "${GRAYLOG_PASSWORD_SECRET:?Please configure GRAYLOG_PASSWORD_SECRET in the .env file}"
      GRAYLOG_ROOT_PASSWORD_SHA2: "${GRAYLOG_ROOT_PASSWORD_SHA2:?Please configure GRAYLOG_ROOT_PASSWORD_SHA2 in the .env file}"
      GRAYLOG_HTTP_BIND_ADDRESS: "0.0.0.0:9000"
      GRAYLOG_HTTP_EXTERNAL_URI: "http://localhost:9000/"
      GRAYLOG_MONGODB_URI: "mongodb://mongodb:27017/graylog"
    ports:
      - "5044:5044/tcp" # Beats
      - "5140:5140/udp" # Syslog
      - "5140:5140/tcp" # Syslog
      - "5555:5555/tcp" # RAW TCP
      - "5555:5555/udp" # RAW UDP
      - "9000:9000/tcp" # Server API
      - "12201:12201/tcp" # GELF TCP
      - "12201:12201/udp" # GELF UDP
      - "10000:10000/tcp" # Custom TCP port
      - "10000:10000/udp" # Custom UDP port
      - "13301:13301/tcp" # Forwarder data
      - "13302:13302/tcp" # Forwarder config
    networks:
      - log
    volumes:
      - "graylog_data:/usr/share/graylog/data/data"
    restart: "on-failure"

networks:
  log:
    driver: "bridge"
    name: log-network

volumes:
  mongodb_data:
  mongodb_config:
  graylog-datanode:
  graylog_data:
```

• Ejecución

```
docker compose up -d
```

• Verificación

```
docker logs log-graylog-1
Initial configuration is accessible at 0.0.0.0:9000, with username 'admin' and password 'QadtbPdtstu'.
Try clicking on http://admin:QadtbPdtstu@0.0.0.0:9000
```

• Generar CA

Configure Certificate Authority

In this first step you can either upload or create a new certificate authority.
Using it we can provision your data nodes with certificates easily.

[Create new CA](#) [Upload CA](#)

Here you can quickly create a new certificate authority. All you need to do is to click on the "Create CA" button. The CA should only be used to secure your Graylog data nodes.

Organization Name *

Graylog CA

Create CA

• Iniciar Sesión: admin/admin1234

- Configurar System -> Input: GELF UDP: recibir y procesar logs provenientes de diferentes fuentes externas

Launch new *GELF UDP* input

☒ Global
Should this input start on all nodes

Title

Bind address

Address to listen on. For example 0.0.0.0 or 127.0.0.1.

Port

Port to listen on.

- Configurar Stream: Se utiliza para clasificar, filtrar y enrutar los mensajes de log que llegan al sistema.

Routing
Launch
Diagnosis

- Select a destination Stream to route messages from this input to.
- **We recommend creating a new stream for each new input.** This will help categorise your messages into a basic schema.
- Messages that are not routed to any Stream will be routed to the **Default Stream**.
- Pipeline rules can be automatically created and attached to the **Default Stream** by this Wizard. These rules will be placed in the system managed Default Routing Pipeline, and will be automatically renamed (or deleted) to accurately reflect the state of this Input.

Route to a new Stream

Recommended!

Route to an existing Stream

- compose.yaml

```
name: app

x-gelf: &gelf
driver: gelf
options:
  gelf-address: "udp://192.168.1.41:12201"

services:
  backend:
    image: ghcr.io/trynewroads/docker-course-advance:latest
    logging:
      <<: *gelf
    environment:
      USE_DB: "true"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASS: password
      DB_NAME: postgres
    ports:
      - "3100:3000"
    depends_on:
      postgres:
        condition: service_healthy

  postgres:
    image: postgres:15

    logging:
      <<: *gelf
    environment:
      POSTGRES_PASSWORD: password
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 20s
      timeout: 10s
      retries: 3

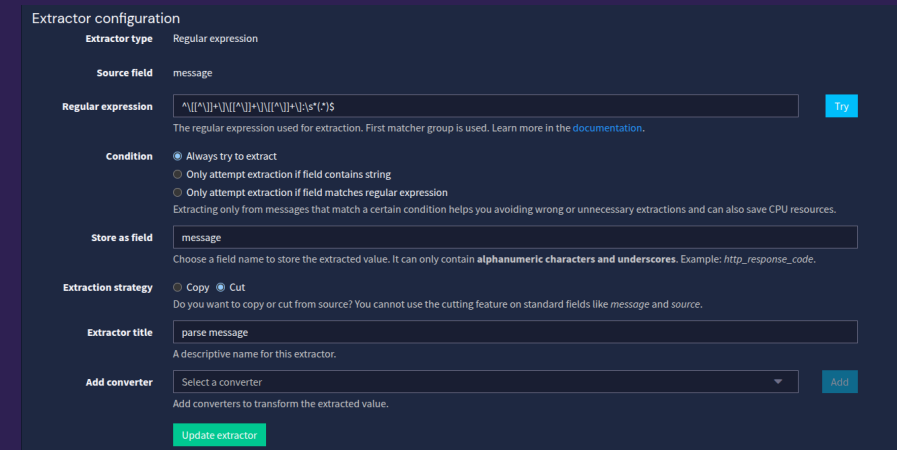
networks:
  default:
    name: app-network
```

- Ejecución

```
docker compose -f 06-log-metric/ejemplos/3.compose/compose.yaml up -d
```

- Verificación: En Graylog podemos ver los mensajes.
- Extraer solo el mensaje:

```
^\\[[^\\]]+\\]\\[[^\\]]+\\]\\[[^\\]]+\\]:\\s*(.*)$
```



The image shows the 'Extractor configuration' window in Graylog. It is configured with the following settings:

- Extractor type:** Regular expression
- Source field:** message
- Regular expression:** `^\\[[^\\]]+\\]\\[[^\\]]+\\]\\[[^\\]]+\\]:\\s*(.*)$` (with a 'Try' button)
- Condition:** ☒ Always try to extract, ☐ Only attempt extraction if field contains string, ☐ Only attempt extraction if field matches regular expression. A note states: 'Extracting only from messages that match a certain condition helps you avoiding wrong or unnecessary extractions and can also save CPU resources.'
- Store as field:** message (with a note: 'Choose a field name to store the extracted value. It can only contain alphanumeric characters and underscores. Example: http_response_code.')
- Extraction strategy:** ☐ Copy, ☒ Cut (with a note: 'Do you want to copy or cut from source? You cannot use the cutting feature on standard fields like message and source.')
- Extractor title:** parse message (with a note: 'A descriptive name for this extractor.')
- Add converter:** Select a converter (with an 'Add' button and a note: 'Add converters to transform the extracted value.')
- Update extractor:** (green button)

- Comprobar

```
curl http://localhost:3000
```

Metrics

Metrics

El comando `docker stats` permite monitorear en tiempo real el uso de recursos de los contenedores.

Para un monitoreo real y centralizado de métricas en entornos productivos, es recomendable integrar soluciones especializadas como **Prometheus**, **Grafana** o herramientas de observabilidad que permitan recolectar, almacenar y visualizar métricas históricas, establecer alertas y analizar tendencias de uso de recursos en toda la infraestructura Docker.

Prometheus

Es una herramienta de monitoreo y almacenamiento de series temporales ampliamente utilizada para recolectar métricas de sistemas y aplicaciones, incluyendo contenedores Docker.

- Permite recolectar métricas de uso de CPU, memoria, red, disco y más, de forma automática y continua.
- Se integra fácilmente con Docker a través de **exporters** como **cAdvisor**, que expone métricas de todos los contenedores en el host.
- Las métricas recolectadas pueden visualizarse y analizarse en tiempo real usando **Grafana**.

```
name: metric

services:
  prometheus:
    container_name: prometheus
    image: prom/prometheus
    volumes:
      - "/prometheus.yml:/etc/prometheus/prometheus.yml"
    ports:
      - "9090:9090"
    networks:
      - metric

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
    ports:
      - "8080:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
    networks:
      - metric

  grafana:
    image: grafana/grafana
    container_name: grafana
    ports:
      - "3000:3000"
    networks:
      - metric
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin

networks:
  metric:
    driver: "bridge"
    name: metric-network
```

- Ejecución

```
docker compose -f 06-log-metric/ejemplos/4.metrics/compose.yaml up -d
```

- Verificación: Acceder <http://localhost:9090/targets>

- Query: podremos realizar las consultas

```
container_memory_usage_bytes{name=~"app-backend-1|app-postgres-1"}
```



- Configurar Prometheus en Grafana: Connections -> Add New connection: <http://prometheus:9090>
- Importamos un dashboard o creamos uno nuevo: <https://grafana.com/grafana/dashboards/193-docker-monitoring/>

Administrador

Portainer

Es una herramienta de gestión visual para entornos Docker y Kubernetes. Permite administrar contenedores, imágenes, volúmenes, redes y stacks desde una interfaz web intuitiva, sin necesidad de usar la línea de comandos.

- Facilita la creación, supervisión y eliminación de contenedores y servicios.
- Permite gestionar múltiples hosts Docker desde un solo panel.
- Ofrece control de acceso por usuarios y roles.
- Ideal para entornos de desarrollo, pruebas y producción.

- compose.yaml

```
services:
  portainer:
    container_name: portainer
    image: portainer/portainer-ce:latest
    restart: always
    networks:
      - 1swarm_swarm-network
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    ports:
      - 9443:9443
      - 8000:8000

volumes:
  portainer_data:
    name: portainer_data

networks:
  default:
    name: portainer_network
  1swarm_swarm-network:
    external: true
```

- Ejecución

```
docker compose -f 06-log-metric/ejemplos/5.portainer/compose.yaml up -d
```

- Acceder <https://localhost:9443>